

★ Master Claude: Cowork, Code, Skills & Plugins

Complete Deep Learning Resource Guide — 15 Sections • 146 Lectures • 22h 25min • 4.6★ Bestseller on Udemy

Compiled from publicly visible course curriculum • All 60+ resource links are free & publicly accessible

Table of Contents

1. [Course Introduction — Claude Ecosystem Overview](#)
2. [Learn Claude Cowork — Browser AI Automation](#)
3. [Claude Skills Basics — Reusable AI Behaviors](#)
4. [Claude Skills Advanced — Expert Design Patterns](#)
5. [Master Claude Plugins — MCP & Tool Integrations](#)
6. [Claude Code Basics — Terminal AI Coding Agent](#)
7. [Claude Code Advanced — Sub-Agents, CLAUDE.md & Deployment](#)
8. [Mastering Claude Design — AI UI/UX Prototyping](#)
9. [Claude Interface & Core Features — Projects, Models, Artifacts, LLMs](#)
10. [Other Claude Concepts — Office, Hidden Features, Migration](#)
11. [Claude With Other Tools — Excalidraw, ElevenLabs, AI Video](#)
12. [Prompt Engineering Basics — Zero/One/Few-Shot & The Formula](#)
13. [Advanced Prompt Engineering — CoT, Reflexion, APE, Context Mgmt](#)

Section 1: Course Introduction

6 lectures • 18 min — Understand all four Claude tools before diving in

Claude Ecosystem: Cowork vs Code vs Skills vs Plugins

Four tools, four distinct purposes: **Claude Cowork** controls your browser/computer autonomously (clicking, typing, form-filling, file management — no coding needed). **Claude Code** is a CLI agent that writes, runs, debugs and deploys real code from your terminal. **Claude Skills** are saved system-prompt instruction sets you reuse across sessions (like behavior presets). **Claude Plugins** use Anthropic's MCP protocol to connect Claude to external APIs — search, calendar, databases, code execution.

COURSE LECTURES

- Explore the Difference Between Claude Cowork, Code, Skills and Plugins (5:19)
- Discover 4 Amazing Use Cases for Claude Cowork & Code Before We Start (6:43)
- How to Get the Most Out of This Masterclass (1:33)
- Download 4 Powerful Claude Gifts + Recommended Tools & Free Resources (3:17 / 0:57)

DEEP LEARNING RESOURCES

[!\[\]\(339a16584d5da0f0a3ca4e9ec17bf6a1_img.jpg\) Claude Models Official Docs](#)[!\[\]\(a870788d6ed9b8fd294b7654a8c8526b_img.jpg\) Claude Code Overview](#)[!\[\]\(de95854c7ee024cfadc48187bbb781b2_img.jpg\) Model Context Protocol \(MCP\)](#)[!\[\]\(3211b5d1d968fc1665909b34f9f16010_img.jpg\) YouTube: Claude AI Full Overview](#)[!\[\]\(6059a5aa8b4ca7bb793408023d6c6e42_img.jpg\) Anthropic Blog: Claude Overview](#)[!\[\]\(c50c8b7b2cc2cf9ff925edec0ee94c0d_img.jpg\) GitHub: Anthropic Cookbook](#)

Section 2: Learn Claude Cowork

19 lectures • 2hr 42min — Control your browser & computer with AI, no coding needed

What Is Claude Cowork & Account Setup

Claude Cowork is a desktop + browser automation platform that gives Claude the ability to act as your computer operator: it can open any website, navigate pages, fill in forms, click buttons, extract text, upload files, and manage your desktop — all from natural language instructions. It uses Anthropic's Computer Use API under the hood. Setup: create account at klot.ai, download the desktop app, connect Chrome extension. Pro: ~\$17/month.

COURSE LECTURES

- What Is Claude Cowork? Full Overview (9:00)
- Claude Cowork vs Claude Code — What's the Difference? (5:42)
- How to Set Up Your Account & Pricing (2:36)

DEEP LEARNING RESOURCES

 [Anthropic Computer Use Official Docs](#)

 [GitHub: Computer Use Demo App](#)

 [YouTube: Claude Computer Use Demo](#)

 [Anthropic: Computer Use Announcement](#)

Building Your First Automation — Use Cases

First automation walkthrough: instruct Claude to open a browser, go to a website, extract specific data, and organize it into a document. Key use cases: web scraping without code, automated form submission, scheduled content publishing, email drafting, competitive monitoring dashboards, and file organisation. Claude reads the actual screen and acts like a human operator.

COURSE LECTURES

- Your First Project with Claude Cowork Step-by-Step (10:50)
- Exercise 1: Use Claude Cowork in Action (3:20)
- Additional Powerful Use Cases of Claude Cowork (6:58)
- 12+ Creative Use Cases You Haven't Thought Of (4:00)

DEEP LEARNING RESOURCES

 [Anthropic: Agents & Computer Use Guide](#)

 [GitHub: Playwright Browser Automation](#)

 [YouTube: AI Browser Automation Workflows](#)

 [n8n: AI Automation Templates](#)

 [Zapier: AI Automation Guide](#)

Chrome Extension & Google Group Tabs

The Claude Chrome extension lets Claude interact with whichever page you're currently viewing — reading page content, filling forms, clicking elements, and publishing responses — without switching apps. Google Group Tabs mode lets Claude work across a set of pinned browser tabs simultaneously, ideal for research, comparison, and multi-site data entry tasks.

COURSE LECTURES

- Use Claude Cowork in Your Browser + Chrome Extension (16:06)
- How to Use Google Group Tabs with Claude Chrome Extension (10:04)

DEEP LEARNING RESOURCES

[📄 Chrome Extension Developer Guide](#)[📺 YouTube: Chrome AI Extensions](#)[🌐 Chrome Content Scripts Docs](#)

Claude Computer Use API & Remote Dispatch from Smartphone

The Anthropic Computer Use API is the technical layer behind Cowork: Claude takes a screenshot, analyses the UI, then sends mouse-click coordinates and keyboard input. Understanding this enables custom deployments. Claude Dispatch lets you send task instructions from your phone and watch Claude execute them on your desktop — fully remote, hands-free operation.

📄 COURSE LECTURES

- Explore Claude Computer Use (20:44)
- Claude Dispatch: Use Claude Cowork From Smartphone (13:01)

🔗 DEEP LEARNING RESOURCES

[📄 Computer Use API Technical Reference](#)[👤 GitHub: All Anthropic Quickstarts](#)[📺 YouTube: Remote AI Automation Setup](#)[🌐 Anthropic Model Specification](#)

Projects, Scheduled Tabs & No-Code Web App Creation

Cowork Projects group related automations with shared context. Scheduled Tabs run automations on a timer (daily, hourly, on trigger) without any manual action — ideal for reports, social posting, price monitoring. The bonus demo builds a complete, deployed web app using only natural language prompts in under 20 minutes, demonstrating no-code app creation.

📄 COURSE LECTURES

- How to Use Claude Cowork Projects & Scheduled Tabs (10:30)
- (Optional) Create & Publish a Web App with Claude Cowork in 20 Minutes! (20:21)

🔗 DEEP LEARNING RESOURCES

[📄 Vercel Deployment Docs](#)[📺 YouTube: AI Workflow Scheduling](#)[🌐 Make.com: Scheduling Automations](#)[👤 GitHub: Workflow Automation Examples](#)

Section 3: Claude Skills Basics

11 lectures • 1hr 28min — Create, test, and manage reusable AI behavior templates

What Are Claude Skills & Core Fundamentals

Claude Skills are saved system-prompt instruction sets stored in your Claude account. When activated, they change how Claude responds for that session — setting a persona, output format, workflow steps, tone rules, or brand guidelines. Think of them as reusable "behavior presets" or custom GPTs, but for Claude. They are accessed via the Customize tab and can be shared or uploaded.

COURSE LECTURES

- Introduction to Claude Skills + 2 Exclusive Gifts (14:38)
- Claude Skills Fundamentals — What You Need to Know (17:24)
- Exercise: Use Skills in Your Claude Account (2:52)
- 20+ Ready-to-Use Skills Pack (1:51)

DEEP LEARNING RESOURCES

 [Anthropic Prompt Engineering Guide](#)

 [System Prompts Reference](#)

 [YouTube: Claude Custom Instructions Deep Dive](#)

 [GitHub: 200+ Awesome Prompts](#)

 [Anthropic: Claude's Character](#)

3 Ways to Create a Skill: Templates, Manual & AI-Assisted

Method 1 — Templates: Use pre-built frameworks for common use cases (customer support bot, writing assistant, code reviewer, data analyst). **Method 2 — Manual from scratch:** Write your own system prompt defining role, task, format, and constraints. **Method 3 — AI-assisted creation:** Describe what you want to Claude, and it generates the skill instructions for you, which you then refine iteratively.

COURSE LECTURES

- 3 Ways to Create a Skill in Claude (4:55)
- How to Create Skills with a Ready-Made Template (11:54)
- Creating a Skill Manually from Scratch (22:36)

DEEP LEARNING RESOURCES

 [Anthropic: System Prompts Reference](#)

 [GitHub: Anthropic Prompt Engineering Tutorial](#)

 [LearnPrompting: Writing Instructions](#)

 [YouTube: Building AI Personas with System Prompts](#)

 [Anthropic: XML Tags in Prompts](#)

Good vs Bad Instructions, Testing Methods & Folder Structure

The quality of skill instructions directly determines output quality. **Bad:** vague directives ("be helpful", "write well"). **Good:** specify role, context, exact task, output format, examples of desired output, and explicit constraints ("never use bullet points", "always respond in JSON"). Three testing methods: (1) Fresh session test — verify from scratch, (2) Functional test — consistent structured output, (3) Value test — real-world usefulness. Organize skills in named folders by category for maintainability.

COURSE LECTURES

- How to Write Good Instructions — Good vs Bad (14:21)
- Testing Your Skills — Three Methods (7:34)
- Good vs Bad Folder Structure (6:18)
- How to Edit and Improve Your Skills (6:59)

DEEP LEARNING RESOURCES

 [Anthropic: Prompt Best Practices](#)

 [Anthropic: Evaluation Guide](#)

 [PromptingGuide: Instruction Basics](#)

 [YouTube: Evaluating Prompt Quality](#)

 [GitHub: Anthropic Evals Framework](#)

Section 4: Claude Skills Advanced

8 lectures • 1hr 16min — Expert-level skill design: conditional logic, chaining, parameterization

Advanced Skill Design: Conditional Logic, Chaining & Parameterization

Advanced patterns beyond simple prompts: **Conditional instructions** — "If the user mentions a budget, provide three pricing tiers; otherwise, describe features first." **Skill chaining** — one skill invokes a workflow where the output of step 1 feeds into step 2 (e.g., research → summarize →

format as slide deck). **Parameterized skills** — skills that adapt dynamically based on user input variables. **External context injection** — embedding brand guidelines, product catalogs, or legal constraints into skill definitions. Also covers organizing skills at scale for team-wide deployment.

COURSE LECTURES

- Advanced Skill Design Patterns
- How to Customize and Chain Skills
- Conditional Logic and Multi-Step Skill Workflows
- Exercise: Create Your Own Skill (2:40)
- Claude Skills Advanced Practice Test 3: Design, Instructions & Testing

DEEP LEARNING RESOURCES

 [Anthropic: Chain-of-Thought Prompting](#)

 [Anthropic: Extended Thinking](#)

 [GitHub: Anthropic Cookbook](#)

 [YouTube: Advanced Claude Prompting Techniques](#)

 [PromptingGuide: Tree of Thoughts](#)

 [GitHub: Prompt Engineering Tutorial](#)

Section 5: Master Claude Plugins

9 lectures • 50min — Connect Claude to external APIs & services via Model Context Protocol

What Are Plugins & Built-In Anthropic Options

Claude Plugins use Anthropic's **Model Context Protocol (MCP)** to connect Claude to external tools and real-time data. Unlike Skills (internal behavior only), Plugins add new capabilities: live web search, code execution, database queries, calendar access, file system operations, weather data. Anthropic's built-in plugins include interactive data/HTML dashboards, marketing tools, customer support templates, HR automation, and PDF generation.

COURSE LECTURES

- What Are Claude Plugins? Full Overview (7:50)
- Claude Plugins vs. Skills — What's the Difference? (1:55)
- How to Use a Plugin Inside Claude (7:22)
- Exercise: Use a Plugin and Share Your Results (1:46)
- Anthropic's Built-In Plugins — Full Overview (4:00)

DEEP LEARNING RESOURCES

[Anthropic: Tool Use Overview](#)[Model Context Protocol \(MCP\) Official Docs](#)[GitHub: MCP Servers Collection \(100+ servers\)](#)[YouTube: Claude Tool Use Tutorial](#)[Anthropic: MCP Announcement Blog](#)

Creating, Customizing & Publishing Your Own Plugins

Building a plugin step-by-step: (1) Define the tool schema in JSON — name, description, input parameters with types, and output format. (2) Implement the backend function or REST API endpoint that executes the tool. (3) Test the integration with Claude sending requests and receiving structured responses. (4) Publish via GitHub URL or direct upload so others can install it. MCP standardizes this interface across different AI assistants.

COURSE LECTURES

- How to Customize Claude Plugins (5:12)
- How to Upload Plugins via GitHub or URL (1:55)
- How to Create Your Own Plugin with Claude (18:00)
- Exercise: Build and Publish Your Very Own Plugin (1:31)

DEEP LEARNING RESOURCES

[Anthropic: Tool Schema Reference](#)[MCP: Building Tools Guide](#)[GitHub: Awesome MCP Servers](#)[YouTube: Build Claude Plugins from Scratch](#)[MCP: Quickstart Server Tutorial](#)

Section 6: Claude Code Basics

23 lectures • 2hr 7min — Install and use Claude's AI terminal agent to build real software

What Is Claude Code & Installation (macOS & Windows)

Claude Code is Anthropic's official CLI AI agent — run it in your terminal and it can read/write any file in your project, execute shell commands, install packages, run tests, commit to Git, and browse documentation — all from natural language. It is NOT a web interface; it has full local

system access. Install: `npm install -g @anthropic-ai/claude-code` (requires Node.js 18+). Works on macOS, Windows (via Git Bash + PowerShell), and Linux.

COURSE LECTURES

- Introduction — What is Claude Code (3:13)
- Claude Code Installation (macOS) (6:59)
- Claude Code Installation (Windows) (10:45)
- Claude Desktop App (1:33)
- A Quick Tour of the Claude Code Interface (6:39)
- Opening Claude Code in Project & Subscription vs API Key (6:28)

DEEP LEARNING RESOURCES

 [Claude Code: Official Getting Started Guide](#)

 [Claude Code: Installation Guide](#)

 [Claude Code: Full CLI Reference](#)

 [YouTube: Claude Code Full Tutorial](#)

 [YouTube: Claude Code Windows Setup](#)

 [GitHub: Claude Code Repository](#)

Git & GitHub — Version Control Fundamentals

Git is the distributed version control system Claude Code uses automatically to track every file change. GitHub hosts remote repositories and enables collaboration and CI/CD pipelines. Even though Claude Code runs Git for you, understanding it is critical for reading its actions and handling conflicts. Key commands: `git init`, `git clone`, `git add`, `git commit`, `git push/pull`, branching, merging, rebasing, and pull requests.

COURSE LECTURES

- Introduction to Git & GitHub (5:06)
- Git vs GitHub — What's the Difference (3:52)

DEEP LEARNING RESOURCES

 [Pro Git Book — Free, Complete Reference](#)

 [GitHub: Hello World Quickstart](#)

 [YouTube: Git & GitHub for Beginners \(freeCodeCamp\)](#)

 [Atlassian: Complete Git Tutorials](#)

 [LearnGitBranching: Interactive Visual Tutorial](#)

IDEs: VS Code & Cursor Setup for Claude Code

An IDE provides the code editor, file explorer, integrated terminal, and syntax highlighting where Claude Code operates. **VS Code** is the most popular, free editor. **Cursor** is an AI-native fork of VS

Code with built-in completions and a chat panel — highly recommended with Claude Code. Key workflow: open project folder in VS Code/Cursor → open integrated terminal → type `claude` → give natural language instructions.

COURSE LECTURES

- Understanding IDEs (4:25)
- Setting Up Claude Code in Cursor (4:37)
- CC in VS Code & Basic Commands (13:27)
- Exercise: Get Comfortable with Claude Code in VS Code (0:31)

DEEP LEARNING RESOURCES

 [VS Code Official Documentation](#)

 [Cursor IDE Documentation](#)

 [YouTube: VS Code Complete Guide](#)

 [YouTube: Cursor AI IDE Tutorial](#)

Building Your First App & Getting Better Results

The core Claude Code loop: describe your requirements in natural language → Claude writes the code files → Claude runs the code → Claude reads any errors → Claude fixes them → repeat until working. Key tips for better results: be specific about tech stack and constraints, reference existing files with `@filename`, use CLAUDE.md for project context, break complex tasks into milestones, and verify outputs at each step.

COURSE LECTURES

- How to Get Better Results from Claude Code (5:10)
- Building Your First App with Claude Code (16:06)
- Exercise: Build a Simple Personal Portfolio Page (0:44)
- Claude Code Basics Practice Test 5: Foundations & Workflow

DEEP LEARNING RESOURCES

 [Claude Code: Official Best Practices](#)

 [Claude Code: Costs & Billing](#)

 [YouTube: Build a Full App with Claude Code](#)

 [Claude Code Troubleshooting Guide](#)

 [GitHub: Claude Code Examples](#)

Section 7: Claude Code Advanced

Advanced Git & GitHub Workflows

Production-level Git with Claude Code: feature branching strategy (create branch → develop → PR → merge), resolving merge conflicts automatically, interactive rebase for clean commit history, semantic versioning for releases, GitHub Actions for CI/CD pipelines (auto-test on push, auto-deploy on merge). Claude Code handles all of this: `gh pr create`, `git rebase -i`, `git tag v1.0.0` — just ask in natural language.

COURSE LECTURES

- Git & GitHub Advanced Walkthrough
- GitHub Authentication & SSH Setup
- Advanced Branching, Rebasing & PR Workflows
- Deploying to GitHub Pages

DEEP LEARNING RESOURCES

[!\[\]\(30072721fe92392a2d7c953be68f714a_img.jpg\) GitHub: Pull Request Collaboration Guide](#)[!\[\]\(3b71157eab31889e641f7620692f0b92_img.jpg\) GitHub CLI \(gh\) Reference](#)[!\[\]\(4d25d87d94191bbe34f0046ad604e903_img.jpg\) GitHub Actions Documentation](#)[!\[\]\(735ceeed4e566aa93749bb6365185b00_img.jpg\) YouTube: Advanced Git Techniques](#)[!\[\]\(67b99c4d964e257c5482ff1b2462b204_img.jpg\) GitFlow: The Branching Model](#)

Sub-Agents: Parallel Task Execution in Claude Code

Claude Code's sub-agent system lets the main "orchestrator" agent spawn multiple specialized "worker" agents running in parallel. Example: build a full-stack web app — orchestrator breaks it into (1) React frontend sub-agent, (2) FastAPI backend sub-agent, (3) pytest test suite sub-agent, all running simultaneously. The orchestrator then synthesizes all outputs. This dramatically reduces build time for complex projects. You can monitor and control which sub-agent handles what.

COURSE LECTURES

- Sub-Agents in Claude Code — Full Architecture Overview
- Running Tasks in Parallel While Keeping Main Agent Focused
- Exercise: Create Your Own Sub-Agent (0:45)

DEEP LEARNING RESOURCES

[!\[\]\(758fecfcf97b15b743a123b5de83ec46_img.jpg\) Claude Code: Sub-Agents Official Guide](#)[!\[\]\(19fdbd6eaa1508fb9caf367b7a64e245_img.jpg\) Anthropic: Building AI Agents](#)[!\[\]\(007bb571fc5331b364ae27ddf8a1c148_img.jpg\) YouTube: Multi-Agent Claude Code Workflow](#)

CLAUDE.md, Skills in Claude Code & Custom Slash Commands

`CLAUDE.md` is the most powerful Claude Code feature: a markdown file at your project root that Claude reads on every startup. Use it to encode codebase conventions, preferred libraries, architecture decisions, authentication patterns, deployment commands, and common workflows — Claude treats it as persistent project memory. Custom slash commands stored in

`.claude/commands/` become one-word shortcuts: `/deploy`, `/test`, `/review`, `/migrate`.

COURSE LECTURES

- Understanding Skills in Claude Code (48:18)
- Exercise: Install and Use a Skill (0:45)
- CLAUDE.md Best Practices
- Custom Slash Command Creation

DEEP LEARNING RESOURCES

 [Claude Code: Memory & CLAUDE.md](#)

 [Claude Code: Slash Commands Reference](#)

 [Claude Code: Settings & Configuration](#)

 [YouTube: CLAUDE.md Best Practices](#)

Full Project Builds: Coffee Shop Website & n8n AI App + Deployment

Two capstone projects: **Project 1:** Complete Coffee Shop website with animated hero sections, menu pages, contact forms, responsive design — fully deployed to production via Vercel. **Project 2:** Integrating n8n (open-source automation platform) to add an AI chat assistant and table reservation system with database storage. Both demonstrate the complete cycle: plan → build → test → debug → deploy — using only Claude Code.

COURSE LECTURES

- Deploying Your Project to the Web (14:47)
- Build a Full Project (Part 1) — Coffee Shop Website & Deployment (1:13:45)
- Build a Full Project (Part 2) — n8n AI Assistant & Reservation System (1:05:42)
- Exercise: Plan, Build, and Deploy a Full Project (1:52)
- 50 Simple Ideas You Can Build with Claude Code — Ready-to-Use Prompts (0:41)

DEEP LEARNING RESOURCES

[Vercel Documentation](#)[Netlify Documentation](#)[n8n Automation Platform Docs](#)[Railway: Deployment Guide](#)[YouTube: Deploy to Vercel Fast](#)[YouTube: n8n Complete Course](#)

Section 8: Mastering Claude Design

12 lectures • 3hr 16min — AI-powered UI/UX design, prototyping, and web deployment

What Is Claude Design & Creating First Projects

Claude Design is Claude's visual-output mode that generates interactive HTML/CSS/JS apps, dashboards, design prototypes, and UI components directly in the browser as Artifacts. Unlike Claude Code (production-grade software), Claude Design optimizes for visual output and rapid iteration — you describe what you want and see a live, interactive result immediately. Perfect for non-developers and designers who need functional prototypes fast.

COURSE LECTURES

- What Is Claude Design? Section Overview (7:10)
- Creating Your First Designs in Claude Design (21:34)
- Exercise: Build Your First Claude Design Project (2:39)

DEEP LEARNING RESOURCES

[Try Claude Design at claude.ai](#)[Anthropic: Artifacts Documentation](#)[YouTube: Claude Artifacts for UI Design](#)[Tailwind CSS Docs — Used in Generated UIs](#)[React Documentation](#)

Building & Applying a Design System

A design system in Claude Design defines: **Design tokens** (primary/secondary colors as CSS variables, type scale, spacing units, border radii), **Components** (buttons with all variants, card layouts, form inputs, modals, navigation bars — all consistently styled), and **Patterns** (page templates, grid layouts). Once Claude has your design system, every subsequent UI it generates automatically follows it — ensuring brand consistency across your entire project.

COURSE LECTURES

- Building a Design System Inside Claude Design (22:07)
- Using Your Design System Across Projects (18:32)
- Claude Design Templates (5:06)

DEEP LEARNING RESOURCES

 [DesignSystems.com — Real-World Design Systems](#)

 [Storybook: Component Documentation](#)

 [Material Design 3 System](#)

 [Atomic Design by Brad Frost](#)

 [YouTube: Design Systems Explained](#)

3D Apps, 6 Use Cases, Full Website Build & Vercel Deploy

Advanced Claude Design outputs: (1) 3D interactive web apps powered by Three.js, (2) Animated data dashboards with D3.js/Chart.js, (3) Mobile app UI prototypes, (4) Marketing landing pages, (5) SaaS dashboards, (6) Portfolio sites. The capstone project builds a complete multi-page website from scratch and deploys it live via Vercel. Also covers token limits, common generation mistakes, how to share/export designs, and the free OpenDesign alternative.

COURSE LECTURES

- Challenge: Create an Interactive 3D Application (3:35)
- Explore 6 Unique Use Cases of Claude Design (13:11)
- Final Project: Build a Complete Website + Deploy with Vercel (41:32)
- Building a Mobile App Prototype (28:22)
- Token Usage, Common Mistakes, Limitations, Sharing Options (25:42)
- Free Alternative to Claude Design — Open Design (6:04)

DEEP LEARNING RESOURCES

 [Three.js Documentation](#)

 [D3.js: Data Visualization Library](#)

 [Vercel: Deploy UI Projects](#)

 [YouTube: Three.js Crash Course](#)

 [YouTube: D3.js Full Tutorial](#)



Section 9: Claude Interface & Core Features

15 lectures • 1hr 17min — Master the claude.ai web interface, models, Projects, and Artifacts

Claude Interface Tour, Models Compared & Pricing

Full walkthrough of claude.ai: chat interface, sidebar Projects, settings, custom instructions, conversation history. **Model comparison:** Claude Haiku — fastest and cheapest, ideal for simple/repetitive tasks. Claude Sonnet — best balance of speed, cost, and capability (current flagship: Sonnet 4.5/3.7). Claude Opus — most powerful reasoning, highest cost, use for complex analysis. **Pricing:** Free (limited messages), Pro (\$20/mo), Max (\$100/mo with higher limits), API (pay-per-token: ~\$3-\$15 per million tokens).

COURSE LECTURES

- Claude's Full Interface Tour (20:20)
- Pricing & Plans Explained (3:31)
- Claude Models & When to Use Which (3:26)
- Get +200 Ready-to-Use Claude Prompts (1:28)

DEEP LEARNING RESOURCES

 [Claude Models Official Overview](#)

 [Anthropic Pricing Page](#)

 [AI Model Benchmarks & Comparisons](#)

 [YouTube: Claude Model Comparison](#)

How LLMs Actually Work — The Technical Foundation

LLMs are **autoregressive transformer models** trained to predict the next token from all previous tokens. Core concepts: **Tokenization** (text split into subword tokens via BPE), **Embeddings** (tokens mapped to dense vector spaces encoding semantic meaning), **Self-attention** (each token attends to all others to build contextual representations), **Multi-head attention** (multiple attention patterns in parallel), **Feed-forward layers** (transform representations), stacked into a deep decoder. Training: pre-train on internet text → RLHF fine-tuning → Constitutional AI (Anthropic's safety method).

COURSE LECTURES

- What is an LLM and How AI Models Actually Work (3:50)

DEEP LEARNING RESOURCES

 [The Illustrated Transformer — Best Visual Explainer](#)

 [YouTube: Andrej Karpathy — Intro to LLMs \(1hr\)](#)

 [YouTube: 3Blue1Brown — Neural Networks Series](#)

 [Paper: Attention Is All You Need \(Original Transformer\)](#)

 [GitHub: nanoGPT — Build GPT from Scratch](#)

 [YouTube: Build GPT from Scratch with Karpathy](#)

Projects, Memory, Artifacts, Voice & Vision Features

Claude Projects: organized workspaces with persistent custom instructions, shared file uploads (PDFs, code, spreadsheets), and knowledge bases visible to all conversations in the project.

Memory: Claude's context window is ~200K tokens — all prior conversation fits until the limit. Context management tips covered. **Artifacts:** self-contained rendered outputs (interactive React apps, HTML tools, SVGs, markdown documents) shown in a live side panel — can be edited, forked, and deployed. Also covers voice input, image/PDF analysis, and file processing.

COURSE LECTURES

- System Prompts & Custom Instructions (3:16)
- Learn How to Use Claude Projects — Deep Dive (8:17)
- Memory — How It Works & How to Use It (4:08)
- Artifacts — Build Things Inside Claude (12:52)
- Exercise 2: Use Artifacts to Build Interactive Interfaces (1:50)
- Prompt Engineering for Real Results (6:17)
- Advanced Features: Voice, Vision, File Analysis (3:04)

DEEP LEARNING RESOURCES

 [Anthropic: Claude Projects Guide](#)

 [Anthropic: Context Windows Explained](#)

 [Anthropic: Vision & Image Analysis](#)

 [Anthropic: Files API](#)

 [YouTube: Claude Projects Masterclass](#)

Section 10: Other Claude Concepts

7 lectures • 59min — PowerPoint/Excel automation, hidden features, migrating from ChatGPT

Claude in PowerPoint & Excel — Office Automation

Claude can automate Microsoft Office tasks through Skills and direct generation: **PowerPoint** — generate full slide decks from outlines, apply brand templates (logos, font styles, color schemes),

create speaker notes, and summarize documents into presentations. **Excel** — clean messy data, build complex formulas, create pivot table summaries, generate charts, run data analysis, and write VBA macros. Skills make these one-click repeatable for recurring tasks like weekly reports.

COURSE LECTURES

- Claude in PowerPoint (9:20)
- Claude in Excel (15:59)
- Skills for Claude in PowerPoint + Excel (6:46)

DEEP LEARNING RESOURCES

 [python-pptx: PowerPoint Automation Library](#)

 [openpyxl: Excel Automation Library](#)

 [YouTube: AI + Excel Automation](#)

 [Microsoft: AI in Excel Guide](#)

Google Group Tabs, ChatGPT Migration & 15 Hidden Features

How to migrate from ChatGPT: export custom GPT instructions → rebuild as Claude Skills, understand behavioral differences (Claude excels at document analysis, longer context 200K tokens, stronger instruction-following). Key advantages of Claude over GPT-4: larger context window, better PDF/document analysis, Constitutional AI safety, superior code generation. **15 hidden features** include: Extended Thinking mode (shows step-by-step reasoning), custom response style, artifact version history, vision analysis of multiple images, Google Drive integration, voice conversations.

COURSE LECTURES

- How to Use Google Group Tabs (with Claude Chrome Extension) (10:04)
- Migrate from ChatGPT to Claude — Import Memory (2:25)
- Why People Switch From Other LLM Models to Claude? (8:17)
- 15 Hidden Features in Claude Nobody is Talking About (6:01)

DEEP LEARNING RESOURCES

 [Anthropic: Claude's Character](#)

 [LMSYS Chatbot Arena: Live Model Rankings](#)

 [YouTube: Claude vs ChatGPT Deep Comparison](#)

 [Anthropic: Feature & Model Changelog](#)

Section 11: Claude With Other Tools

6 lectures • 2hr 5min — Combine Claude with design, diagram, video, voice, and research tools

Claude + Excalidraw — System Diagrams from Natural Language

Excalidraw is an open-source whiteboard tool for technical diagrams. Claude can generate complete Excalidraw diagrams from a description: system architecture diagrams, ER diagrams, flowcharts, sequence diagrams, UML. Claude outputs the Excalidraw JSON format which you paste directly into excalidraw.com — instant, fully editable diagrams in seconds instead of hours of manual drawing.

COURSE LECTURES

- Claude + Excalidraw = Complex Diagrams in Seconds! (25:42)

DEEP LEARNING RESOURCES

 [Excalidraw Documentation](#)

 [YouTube: Excalidraw Complete Tutorial](#)

 [Excalidraw Help Center](#)

 [GitHub: Excalidraw Open Source](#)

Claude + Canva & NotebookLM — Design & Research Workflows

Claude + Canva: use Claude to write ad copy, social media captions, design briefs, and marketing text — then paste into Canva for visual design. Canva's AI can further adapt Claude's copy into visual elements. Claude + NotebookLM (Google): NotebookLM is source-grounded (only answers from your uploaded documents). Use Claude for broad analysis and ideation, NotebookLM for precise citations from specific source documents — a powerful research duo.

COURSE LECTURES

- Claude + Canva = The Ultimate Designer (6:31)
- Claude + NotebookLM (14:04)

DEEP LEARNING RESOURCES

 [Canva AI Design Features](#)

 [Google NotebookLM](#)

 [YouTube: NotebookLM Full Tutorial](#)

Claude + AI Video Generation (Higgsfield, Veo) & Video Editing

End-to-end AI video production pipeline: Claude writes the video script, shot descriptions, and voiceover text → Higgsfield or Veo 3.1 generates video from text prompts → Claude advises on video editing sequences and transitions. Models covered: Seedance 2.0, Nano Banana 2, Veo 3.1, GPT Image 2. Full guide on using Claude to script, structure, and direct entire video productions.

COURSE LECTURES

- Claude + Higgsfield (GPT Image 2, Nano Banana 2, Seedance 2.0, Veo 3.1) (23:13)
- Claude for Video Editing — Full Guide (32:27)

DEEP LEARNING RESOURCES

 [Higgsfield AI Documentation](#)

 [YouTube: AI Video Production Pipeline](#)

 [RunwayML Gen-3 Video](#)

 [GitHub: Google Veo Resources](#)

Claude + ElevenLabs — AI Voice & Podcast Production

ElevenLabs creates ultra-realistic AI voiceovers, cloned voices, and audio content. Workflow: Claude writes the script, blog post, or podcast outline → ElevenLabs generates the voice audio. Perfect for: YouTube narrations, podcast episodes, audiobooks, e-learning content, and product demos. Claude can also write ElevenLabs-optimized scripts (natural speaking pace, emphasis markers).

COURSE LECTURES

- Claude + ElevenLabs (22:54)

DEEP LEARNING RESOURCES

 [ElevenLabs Documentation](#)

 [YouTube: ElevenLabs Full Tutorial](#)

 [ElevenLabs: Voice Lab](#)

Section 12: Prompt Engineering Basics

10 lectures • 58min — *The proven formula for communicating with AI effectively*

What is Prompting & The RCTFC Formula

A prompt is any natural language instruction given to an LLM. Prompt quality directly determines output quality. The proven **RCTFC Formula**: **R**ole — who Claude should be ("You are a senior Python developer"), **C**ontext — relevant background ("I have a Flask app with JWT auth"), **T**ask — the exact thing to do ("Write a token validation function"), **F**ormat — how to structure output ("Return as a code block with inline comments"), **C**onstraints — what NOT to do ("Do not use libraries beyond PyJWT"). Examples of bad vs good prompts demonstrated in detail.

COURSE LECTURES

- What is a Prompt? (5:15)
- Let's Discover What is Prompt Engineering (8:32)
- Learn The Proven Formula For Creating The Best Prompts (6:32)
- Use Perfect Formula in Action (10:44)
- Good Prompt vs Bad Prompt — What is The Difference? (5:55)

DEEP LEARNING RESOURCES

 [Anthropic: Complete Prompt Engineering Guide](#)

 [PromptingGuide.ai — Full Technique Reference](#)

 [YouTube: Prompt Engineering Full Course \(freeCodeCamp, 4hrs\)](#)

 [GitHub: DAIR.AI Prompt Engineering Guide](#)

 [LearnPrompting.org — Interactive Free Course](#)

 [Anthropic: Auto Prompt Generator Tool](#)

Zero-Shot, One-Shot & Few-Shot Prompting

Zero-shot: No examples — just an instruction. Claude relies entirely on training data. Works for well-defined tasks with clear expectations. **One-shot:** One example that sets the pattern and format. Best for style matching. **Few-shot:** 3-10 examples showing exactly what good output looks like. Dramatically improves consistency, format adherence, and accuracy — especially for classification, structured data generation, and specific writing styles. Generally the most reliable prompting technique.

COURSE LECTURES

- Explore Zero-Shot Prompting (5:44)
- One-Shot Prompting (4:25)
- Few-Shot Prompting (5:42)

DEEP LEARNING RESOURCES

 [PromptingGuide: Few-Shot Prompting](#)

 [Paper: GPT-3 Few-Shot Learners \(Brown et al.\)](#)

 [YouTube: Zero, One, Few-Shot Explained](#)

 [SuperAnnotate: Few-Shot Complete Guide](#)

Understanding LLM Models — Technical Depth

LLM architectures: decoder-only (Claude, GPT, Llama) vs encoder-decoder (T5). Model size matters: 7B parameters (fast, local), 70B (balanced), 400B+ (frontier). Key settings: **Temperature** (0=deterministic/factual, 1=creative/varied), **Top-p** (nucleus sampling — controls vocabulary

diversity), **Top-k** (limits token candidates), **Max tokens** (output length cap). Model families: Claude (Anthropic), GPT/o1/o3 (OpenAI), Gemini (Google), Llama (Meta open-source), Mistral (open-source).

COURSE LECTURES

- What are LLM Models? (5:01)

DEEP LEARNING RESOURCES

 [HuggingFace Transformers Documentation](#)

 [YouTube: Karpathy — 1hr LLM Introduction](#)

 [LifeArchitect: LLM Timeline & Comparison](#)

 [GitHub: Awesome LLM Resources](#)

 [Ollama: Run LLMs Locally](#)



Section 13: Advanced Prompt Engineering

8 lectures • 33min — Expert techniques: CoT, Reflexion, APE, dynamic prompting

Mastering Output Control — Format, Persona & Constraints

Expert techniques for precise output control: **Structured output** — ask Claude to respond in JSON/XML with a defined schema (field names, types, nesting). **Persona injection** — detailed role definition changes writing style, expertise level, and knowledge domain. **Negative constraints** — explicit "never" statements prevent unwanted behaviors. **Self-critique loop** — ask Claude to rate its output 1-10 and then improve it. **Style matching** — provide a writing sample and ask Claude to match it exactly. These techniques compound: combining structured output + persona + constraints dramatically improves reliability.

COURSE LECTURES

- Master Techniques to Perfectly Control the Final Response (9:18)

DEEP LEARNING RESOURCES

 [Anthropic: Controlling Output Format](#)

 [Anthropic: Reducing Hallucinations](#)

 [Anthropic: XML Tags in Prompts](#)

 [YouTube: Advanced Output Control Techniques](#)

Chain-of-Thought (CoT) Prompting

Chain-of-Thought is one of the most impactful prompt engineering techniques: adding "think step by step" or "reason through this carefully" makes the model externalize its reasoning process before giving a final answer. This dramatically improves accuracy on math (+30%), logical reasoning (+40%), and multi-step coding tasks. Variants: standard CoT, zero-shot CoT ("let's think step by step"), self-consistent CoT (sample multiple reasoning paths and vote), and Tree of Thoughts (explore multiple branches).

COURSE LECTURES

- Find Out What The Chain-Of-Thought Method is (3:49)
- Use This Method To Handle Complex Prompts (2:19)

DEEP LEARNING RESOURCES

 [PromptingGuide: Chain-of-Thought](#)

 [Paper: Chain-of-Thought Prompting \(Wei et al., 2022\)](#)

 [YouTube: Chain of Thought Deep Dive](#)

 [Tree of Thoughts — Extended CoT](#)

Reflexion Technique

Reflexion (Shinn et al. 2023) is a framework where an agent evaluates its own outputs and iteratively improves them using verbal reinforcement. In practice with Claude: (1) Claude solves the problem, (2) Claude critiques its own answer ("what's wrong with this?"), (3) Claude improves based on the critique. Repeat until satisfactory. Highly effective for: code debugging, essay revision, analysis refinement. Outperforms single-pass generation on complex tasks.

COURSE LECTURES

- Reflexion Technique (4:30)

DEEP LEARNING RESOURCES

 [Paper: Reflexion Framework \(Shinn et al., 2023\)](#)

 [YouTube: Reflexion AI Technique](#)

 [PromptingGuide: Reflexion](#)

Automatic Prompt Engineering (APE), Dynamic Prompting & Context Management

APE: Use Claude to generate optimal prompts for your task — describe your goal, Claude outputs the best system prompt. Iteratively refine. **Dynamic prompting:** Build prompts programmatically with variable injection — user data, retrieved context, template slots filled at runtime (used in production AI apps). **Context management:** For conversations exceeding context limits —

summarize older turns, use sliding windows (keep last N exchanges + summary), hierarchical compression, and context extraction into CLAUDE.md or Projects knowledge.

COURSE LECTURES

- Automatic Prompt Engineering (1:55)
- Dynamic Prompting (3:55)
- Context Management Technique (6:32)

DEEP LEARNING RESOURCES

 [Paper: LLMs as Optimizers — APE](#)

 [Anthropic: Long Context Tips](#)

 [GitHub: Microsoft PromptBase](#)

 [GitHub: LangChain — Dynamic Prompt Chains](#)

 [YouTube: LangChain Complete Course](#)

✦ **About This Guide:** Compiled from the publicly visible Udemy course curriculum of *Master Claude Cowork, Claude Code, Skills & Plugins* (4.6★ Bestseller, 9,181 students). All 60+ resource links are official documentation, free YouTube tutorials, academic papers, and GitHub repos. **To save as PDF:** click the purple button or press Ctrl+P → "Save as PDF". To save as HTML: press Ctrl+S.